Advanced Database Systems Portfolio Summary

Olaotan Leigh | Student ID 301920357 | MIT 8103

2026/2027 academic session | First semester | 10 November 2026

Repository: Leigh-Ola/Leigh_Olaotan_301920357.

Selected case and scope

Harbour Office Supplies is a fictional Nigerian online stationery shop with one stockroom. Customers place orders. Sales staff record orders and full payments.

Stock staff control available quantities and unpaid-order cancellations. The owner reads sales and stock reports. All data is fictional.

All amounts use Nigerian naira (NGN).

The same case supports all five activities. PostgreSQL 17.11 controls customer, product, order, stock and payment data. MongoDB 7.0.43 stores order events. P1 to P4 contain executed local work. P5 contains a proposed distributed design.

Portfolio 1 Database design

Five tables store customers, products, orders, order lines and payments. The design defines 25 attributes. Normalisation examples show the change from repeated item lists to first, second and third normal form. Historical line prices stay fixed when catalogue prices change.

The schema has 31 named constraints and 24 required columns. Its 39-row fixture contains six customers, eight products, eight orders, 13 lines and four payments. All 78 invalid operations failed for the expected reason. Eight valid boundary tests passed. All business rows stayed unchanged after the tests.

Six tests show rules that row constraints cannot enforce alone. P3 adds restricted transaction operations for those rules. An isolated replay matched the business data with different generated IDs. P1 design and evidence.

Portfolio 2 Queries and performance

Six SQL queries answer sales, debt, customer and stock questions. Paid sales total NGN 24,500. Unpaid placed orders total NGN 6,500. The results use historical line prices and keep customers with no orders where appropriate.

An isolated benchmark contains 1,030,100 rows, including 300,000 orders. Each phase uses one warm-up and five measured runs for each query. The ordered results match before and after the index change.

Query	Before median	After median
Selective customer history	19.032 ms	0.578 ms
Broad status count	44.913 ms	43.769 ms

The customer-history index reduces the selective query's work. The broad scan has no useful measured improvement. The index adds 11.62 MiB of storage. Write costs were not measured. The 400 KiB shared buffer pool and unmeasured operating-system cache limit wider performance claims. P2 results and method.

Portfolio 3 Transactions and concurrency

The transaction functions place orders, record full payments and cancel unpaid orders. A restricted practice role can call these functions but cannot change table records directly. Administrators can bypass that boundary. The exercise does not implement customer login or a deployed application.

A committed order reserves two pen packs and one paper ream, with a total of NGN 4,500. An intentional stock error causes rollback of the full failed transaction. A comparison of all five tables shows no partial business changes. The tests also cover payment, cancellation and 25 rejected operations.

Two independent sessions demonstrate four cases. A third connection observes blocking and the final data.

Case	Observed result
Last stock unit	One order succeeds. The waiting order finds zero stock and fails.
Payment first	The waiting cancellation fails after payment commits.
Cancellation first	The waiting payment fails after cancellation commits.
READ COMMITTED visibility	A session sees the new stock value only after the other session commits.

Row locks and stock tests after locking prevent overselling. READ COMMITTED alone does not enforce the shop rules. An isolated replay passed all four cases. P3 scripts and transcripts.

Portfolio 4 NoSQL model

MongoDB stores one document for each order event. Fourteen documents represent eight placements, four payments and two cancellations from P1. Embedded customer and item snapshots make each event readable. BSON dates and Decimal128 values preserve time and money types.

Seven queries passed, including the NGN 24,500 payment total. The history query used an index and examined two documents for two results. This small data set does not support a speed improvement claim. Twenty-one invalid operations failed with the expected error codes.

The validator controls document structure. It does not enforce PostgreSQL foreign keys or full payment across events. It does not reject payment after cancellation or duplicate item SKUs. Tests demonstrate these limits. A replay matched all BSON values and index rules. The event data also matches the P1 business records. There is no automatic database synchronisation. P4 model and results.

Portfolio 5 Distributed database proposal

The proposal keeps PostgreSQL as the business authority. It adds a synchronous standby and a distinct three-member MongoDB replica set across failure zones. A proposed outbox and publisher deliver committed events with safe retries. These components are not implemented.

The discussion covers primary failure, loss of majority access, delayed history, fencing, backups and restore tests. It explains why replication does not replace transaction rules or backups. Fourteen events do not justify sharding. The proposal has no measured cloud performance or failover result.

Reflection

I chose five normalised tables to reduce repeated facts and preserve clear relationships. Historical prices protect earlier sales values. Exact decimal types store money. Transactions and row locks coordinate shared stock. A targeted index supports customer-history queries. MongoDB event documents keep readable snapshots. PostgreSQL controls the business rules. The proposed distributed design uses replication. The small event dataset does not justify sharding.

One challenge was separating row constraints from business rules. A foreign key links an order line to an order. It cannot require every order to contain a line.

Another challenge was that implementing concurrency required me to think carfully about the implications of shared stock. I realized that using READ COMMITTED alone does not prevent overselling. I needed to understand why the workflow checks stock after locking. Payment and cancellation also need the same order lock.

Finally, i realized rgar MongoDB required me to be able to clearly know the difference between document structure and business requirements. For example, MongoDB's validator checks fields and types, but does not compare payments with PostgreSQL totals or enforce any particular event order.

The main technical lesson is the connection between data design and business rules. Normalisation keeps customer, product, order and payment facts in suitable tables. Historical line prices preserve earlier sales values. Row constraints protect individual records. Transactions coordinate stock, payment and cancellation. The index experiment shows why performance claims must use measured plans and unchanged query results.

AI use and verification

OpenAI Codex assisted with design, code, tests, source research, documentation and formatting. GPT-6.1 Sol reviewed earlier work and prepared a reflection draft. AI also helped diagnose errors and compare results.

I selected the small online shop case and directed the database design, work stages and document structure. I combined manual work with AI assistance. I compared the schema with the design, ran valid and invalid operations, and compared the data before and after the tests. I repeated these tests in isolated databases. I reviewed all scripts and results before submission.

The recorded evidence includes query results, index comparisons, session waits, stock values and MongoDB validation tests. The references identify external sources. The portfolio generator was not executed. External library data and results were not submitted as shop evidence. All business data is fictional.

References

1. Miva Open University. MIT 8103 assessment, 2026/2027, pages 3 to 7.

2. PostgreSQL Global Development Group. PostgreSQL 17 documentation. Constraints, plans, indexes, isolation, locks, replication and recovery.

3. MongoDB. MongoDB 7.0 documentation. Modelling, validation, replication, elections, read and write concerns, sharding and backups.

4. arizonal2014. MIT8103 Codelab, revision 619af2b. Work sequence and evidence examples.

5. arizonal2014. Portfolio Generator, revision 7b738ab. Test and report ideas.

6. arizonal2014. Complete library example, revision 4d924b3. Benchmark and session evidence layout.

The repository contains the detailed reference list and links beside technical claims. P5 sources were accessed on 8 November 2026.